



UNIVERSIDADE FEDERAL DE ITAJUBÁ

Algoritmos e Estrutura de Dados I

CTC001

Recursão

Vanessa Cristina Oliveira de Souza



- ▶ Método de **resolução de problemas** que envolve **quebrar um problema em subproblemas menores** até chegar a um problema pequeno o suficiente para que ele possa ser resolvido **trivialmente**.
- ▶ Um método que chama a si mesmo direta, ou indiretamente, é dito **recursivo**.





Recursão

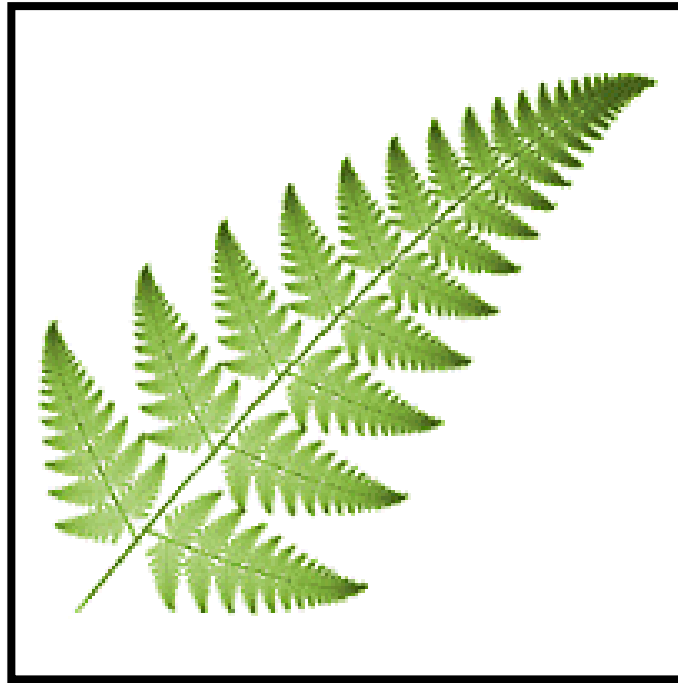


Matrioska





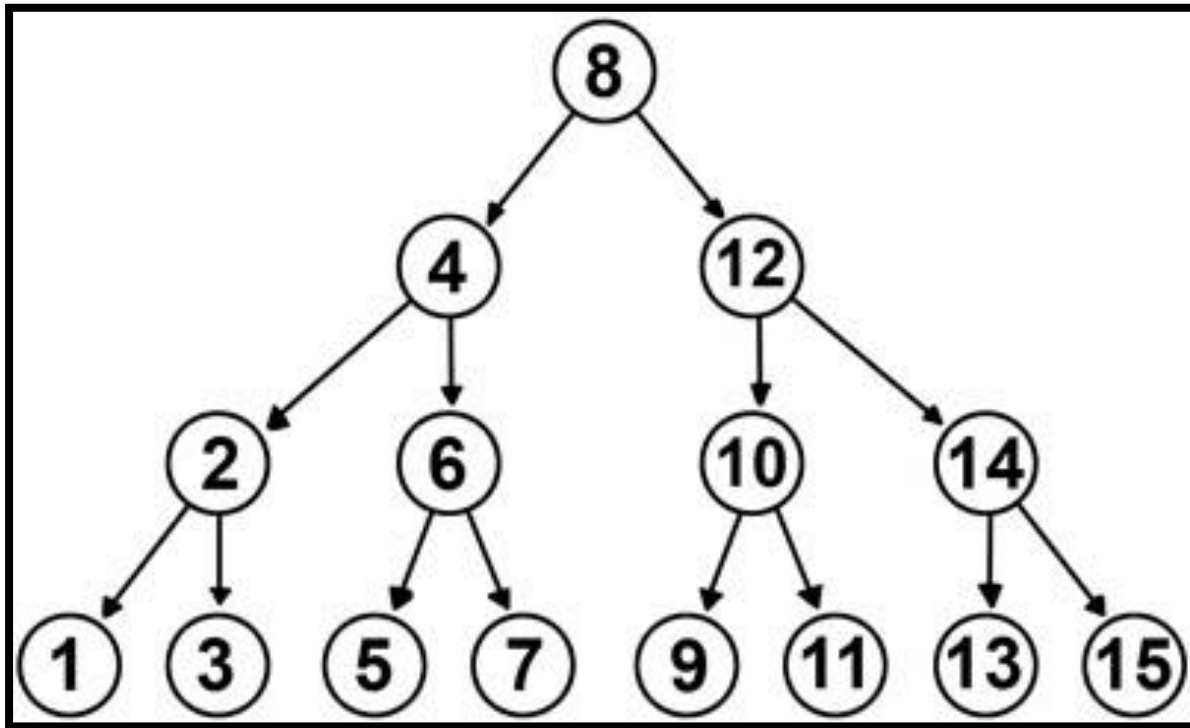
Recursão



Fractais Geométricos



Recursão



Árvore Binária de Busca



Leis da Recursão

- ▶ Todo algoritmo recursivo deve obedecer a 3 leis:
 1. O algoritmo recursivo deve possuir um **caso base**;
 2. O algoritmo recursivo deve alterar o seu estado de maneira a **se aproximar do caso base**;
 3. O algoritmo recursivo deve ter uma **chamada a si mesmo** (direta ou indiretamente).





Como o compilador implementa a recursão?

- ▶ Um compilador implementa um método recursivo por meio de uma **pilha**, na qual são armazenados os dados usados em cada chamada de um método que ainda não terminou de processar.
- ▶ Todos os dados não globais vão para a pilha, pois o **estado corrente da computação deve ser registrado** para que possa ser recuperado posteriormente.





Como funciona a Recursão?

- ▶ Pode-se dizer que os algoritmos recursivos apresentam duas fases:
 - ▶ Fase de expansão
 - ▶ Fase de encolhimento





Como funciona a Recursão?

- ▶ Pode-se dizer que os algoritmos recursivos apresentam duas fases:
 - ▶ **Fase de expansão**
 - ▶ As chamadas da função são **empilhadas**, mas ainda não apresentam resultado algum
 - ▶ As chamadas são empilhadas até que se chegue ao **caso base**
 - ▶ O 'empilhamento de chamadas' acontece quando o método chama a ele mesmo





Como funciona a Recursão?

- ▶ Pode-se dizer que os algoritmos recursivos apresentam duas fases:
 - ▶ **Fase de encolhimento**
 - ▶ Chegando no caso base, é possível começar a realizar a computação
 - ▶ As chamadas começam a ser desempilhadas
 - ▶ A computação termina quando a pilha estiver vazia





Como funciona a Recursão?

- ▶ Pode-se dizer que os algoritmos recursivos apresentam duas fases:
 - ▶ **Fase de encolhimento**
 - ▶ Chegando no caso base, é possível começar a realizar a computação
 - ▶ As chamadas começam a ser desempilhadas
 - ▶ A computação termina quando a pilha estiver vazia

LEMBRETE:

**UMA FUNÇÃO SEMPRE RETORNA O VALOR CALCULADO
PARA AQUELA FUNÇÃO QUE A CHAMOU!**



Exemplos Clássicos

Fatorial



Exemplos Clássicos – Fatorial

- ▶ Calcular o fatorial de um número
 - ▶ $n!$ = Produto de todos os inteiros positivos menores ou iguais a n

$$n! = \prod_{k=1}^n k$$

$$5! = 1 * 2 * 3 * 4 * 5 \Rightarrow 120$$

$$0! = 1$$





Exemplos Clássicos – Fatorial

- ▶ Calcular o fatorial de um número
 - ▶ Solução Iterativa

```
int fatIt(int valor)
{
    int i, fat=1;
    for (i=1;i<=valor;i++)
    {
        fat = fat * i;
    }
    return fat;
}
```





Exemplos Clássicos – Fatorial

- ▶ É possível implementar uma solução recursiva?
 - ▶ O problema apresenta uma característica recursiva? Ou seja, algo que se repete e que vai gerar a chamada a ele mesmo?
- ▶ Qual seria o caso base?
 - ▶ Qual a parte do problema que você sabe resolver trivialmente?
- ▶ Como a gente se aproxima do caso base?
 - ▶ Como você vai diminuindo o problema grande para se aproximar da solução que você conhece?





Exemplos Clássicos – Fatorial

- ▶ Calcular o fatorial de um número
 - ▶ Definição Recursiva

$$n! = \begin{cases} 1, & \text{se } n = 0 \\ n * (n-1)!, & \text{se } n > 0 \end{cases}$$





Exemplos Clássicos – Fatorial

- ▶ Calcular o fatorial de um número
 - ▶ Solução Recursiva

```
1  int fatRecursivo(int valor)
2  {
3      if (valor == 1 || valor == 0)
4          return 1;
5      int res = valor * fatRecursivo(valor - 1);
6      return res;
7  }
```

Caso Base

Chamada
recursiva e
aproximação
do caso base



Exemplos Clássicos – Fatorial

TESTE DE MESA

- ▶ Calcular o fatorial de um número
 - ▶ Solução Recursiva

```
res = fatRecursivo(5);
```

```
1 int fatRecursivo(int valor)
2 {
3     if (valor == 1 || valor == 0)
4         return 1;
5     int res = valor * fatRecursivo(valor - 1);
6     return res;
7 }
```

Caso Base

Chamada
recursiva e
aproximação
do caso base

Exemplos Clássicos

Máximo Divisor Comum (MDC) Recursivo

Algoritmo de Euclides



Exemplos Clássicos – MDC

- ▶ O algoritmo de Euclides para encontrar o máximo divisor comum (MDC) entre dois números.

MDC(24,18)

24	18	





Exemplos Clássicos – MDC

- ▶ O algoritmo de Euclides para encontrar o máximo divisor comum (MDC) entre dois números.

MDC(24,18)

	1	
24	18	
6		





Exemplos Clássicos – MDC

- ▶ O algoritmo de Euclides para encontrar o máximo divisor comum (MDC) entre dois números.

MDC(24,18)

	1	
24	18	6
6		





Exemplos Clássicos – MDC

- ▶ O algoritmo de Euclides para encontrar o máximo divisor comum (MDC) entre dois números.

MDC(24,18)

	1	3
24	18	6
6	0	





Exemplos Clássicos – MDC

- ▶ O algoritmo de Euclides para encontrar o máximo divisor comum (MDC) entre dois números.

$$\text{MDC}(24, 18) = 6$$

	1	3
24	18	6
6	0	





Exemplos Clássicos – MDC

- ▶ O algoritmo de Euclides para encontrar o máximo divisor comum (MDC) entre dois números.
 - ▶ Qual o caso base?
 - ▶ Qual a chamada recursiva?





Exemplos Clássicos

Fibonacci



Exemplos Clássicos – Fibonacci

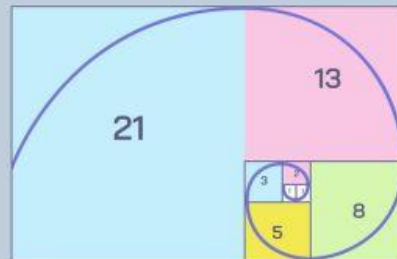
- Calcular a sequência de Fibonacci

A sequência de Fibonacci

Cada número é a soma dos seus dois antecessores

1 1 2 3 5 8 13 21

$$\begin{aligned}1 + 1 &= 2 \\1 + 2 &= 3 \\2 + 3 &= 5 \\3 + 5 &= 8 \\5 + 8 &= 13 \\8 + 13 &= 21\end{aligned}$$



→ Fórmula de Fibonacci

$$F_n = F_{n-1} + F_{n-2}$$



Funções Recursivas vs Funções Iterativas



Comparação Algoritmos Recursivos vs Iterativos

Critério	Iterativo	Recursivo
Modo de implementação	Implementação utilizando loops	Função chama ela mesma
Estado	Definido pelo valor da variável de controle	Definido pelos parâmetros dos valores armazenados na pilha
Progressão	O valor da variável de controle move-se em direção ao valor na condição	O estado da função convirte para o caso base
Terminação	Loop termina quando a variável de controle satisfaz a condição	Recursão termina quando o caso base se torna verdadeiro
Tamanho do código	Tende a ser maior	A recursão diminui o tamanho do código
Sem terminação	O loop infinito usa ciclos de CPU	Recursão infinita pode causar Stack overflow e pode travar o sistema
Execução	A execução é mais rápida e exige menos memória	A execução é mais lenta e exige mais memória





Comparação Algoritmos Recursivos vs Iterativos

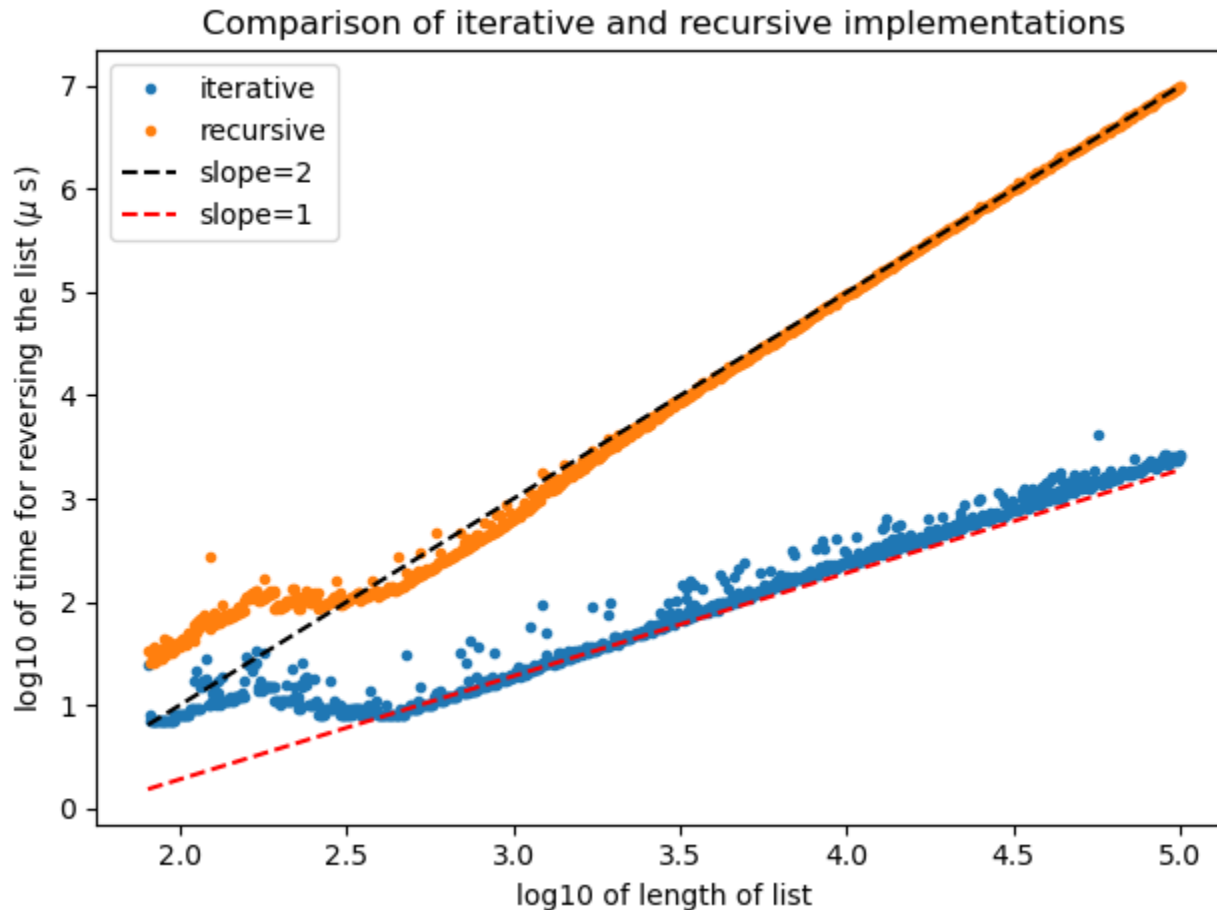
N	Recursion (Time in ms)	Iteration (Time in ms)	Sequence Number (At index N)
5	0	5	5
10	0	5	55
15	1	5	610
20	1	4	6765
25	2	4	75025
30	11	3	832040
35	12	3	9227465
40	101	4	102334155
45	11615	5	1134903170
50	128601	5	12586269025
70	Long time/Stack Overflow	5	190392490709135
90	Long time/Stack Overflow	5	2880067194370816120

<https://medium.com/nerd-for-tech/practically-understanding-time-complexities-of-recursive-and-iterative-functions-95238525c145>





Comparação Algoritmos Recursivos vs Iterativos



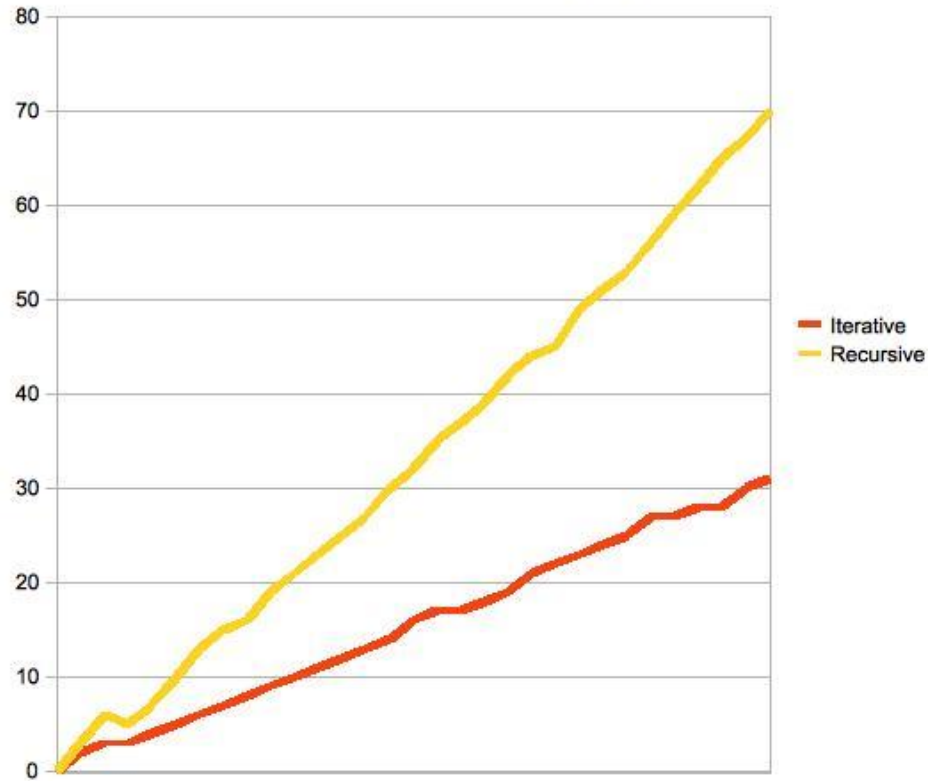
<https://stackoverflow.com/questions/64515062/high-performance-recursive-functions-in-c>



Comparação Algoritmos Recursivos vs Iterativos

Recursion Speed

<http://jacksondunstan.com/articles/957>

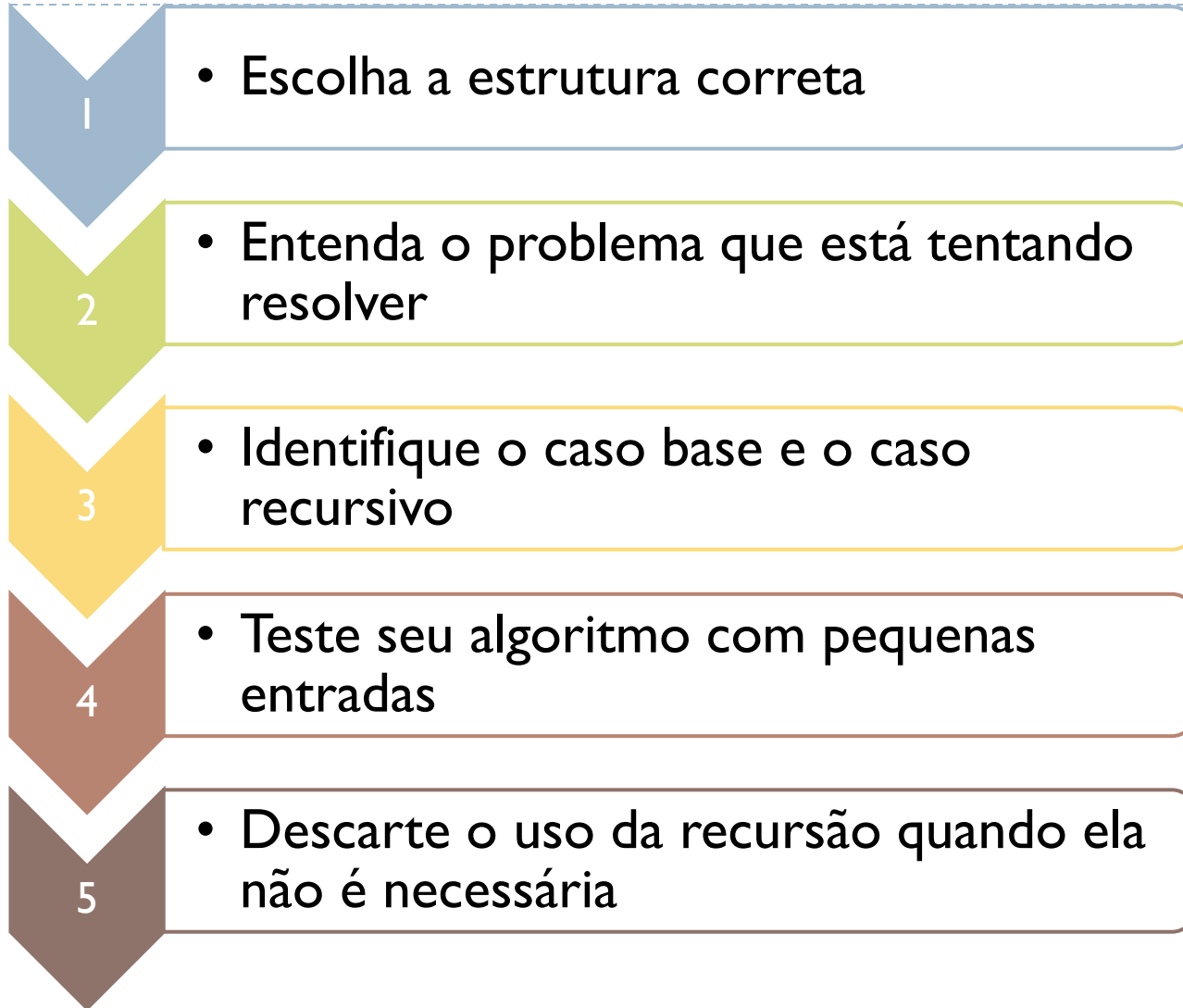


<https://www.jacksondunstan.com/articles/957>





Melhores práticas para escrever algoritmos recursivos





Aplicações Típicas

- ▶ Técnicas dividir para conquistar
- ▶ Algoritmos de *backtracking*
- ▶ Permutações e combinações
- ▶ Buscas





Recursão

